CSE 102 Assignment 5 Report

1. Project Overview

This project is a simplified implementation of the Battleship game in C. The game generates a 10x10 board and randomly places 4 ships of various sizes. Players try to sink all ships by entering coordinates. The program logs all hits to a file named ships.txt, including the position and size of each hit ship segment.

2. Key Logic and Functions

2.1 Board Initialization

```c
void initializeBoards() {
    for (int row = 0; row < BOARD_SIZE; row++) {
        for (int col = 0; col < BOARD_SIZE; col++) {
            hiddenBoard[row][col] = '-';
            playerBoard[row][col] = '-';
            shipMap[row][col] = -1; // no ship assigned yet
        }
    }
}
```

2.2 Ship Placement with Tracking

```c
shipMap[currentRow][currentCol] = shipIndex;
```

When a ship is placed, each cell it occupies is recorded with its corresponding ship index in shipMap.

2.3 Logging Hits to ships.txt

```c
FILE *hitFile = fopen("ships.txt", "a");
if (hitFile != NULL) {
    fprintf(hitFile, "%d %d %d\n", row, col, shipLengths[hitShipIndex]);
    fclose(hitFile);
}
```

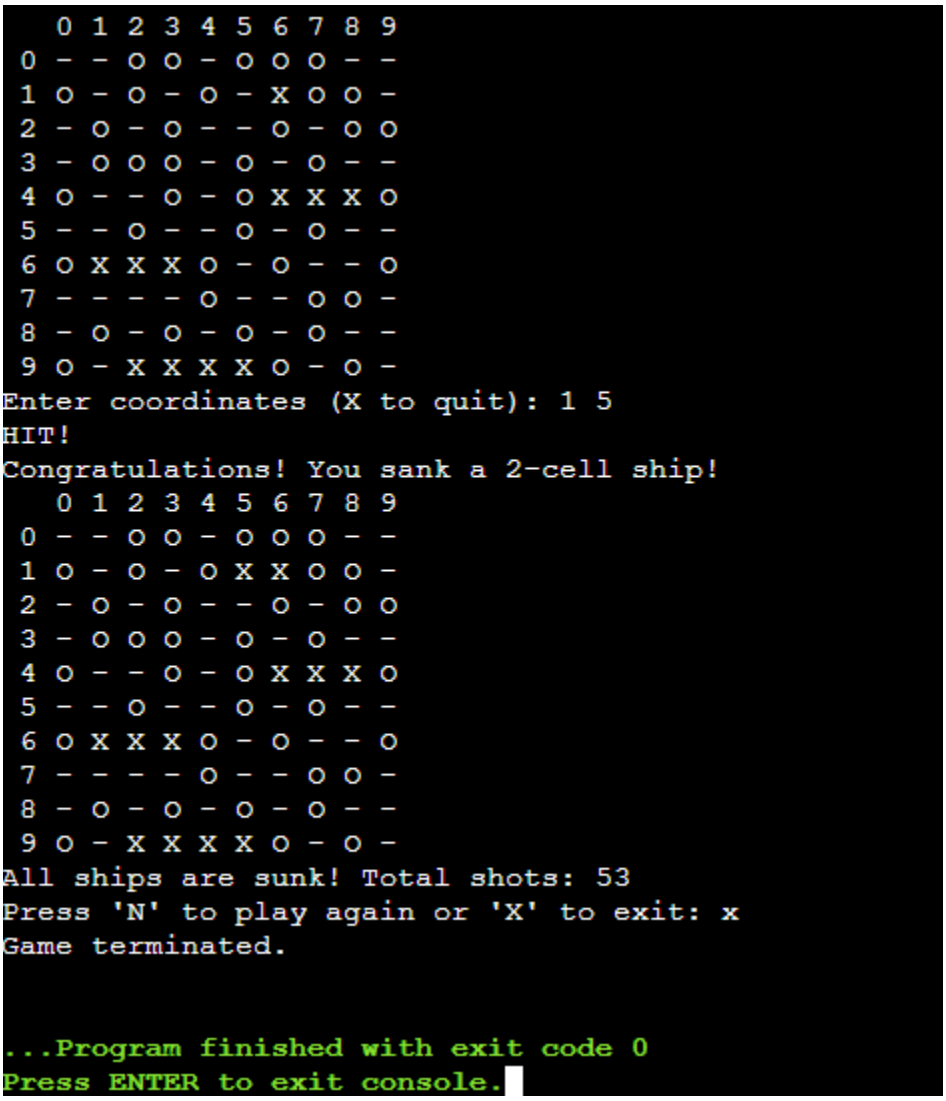
This part ensures that every successful hit on a ship immediately gets written to ships.txt with the format: <row> <column> <ship_length>

3. Observations and Challenges

- Sometimes hit logs were missing because the mapping (shipMap) was not updated properly.

- Two ships with the same length caused confusion in hit tracking when hits were not clearly attributed.
- The issue was resolved by uniquely tagging each cell with a shipIndex, allowing accurate tracking per ship.

4.Outputs



```
   0 1 2 3 4 5 6 7 8 9
 0 - - o o - o o o - -
 1 o - o - o - X - o -
 2 - o - o - - o - o o
 3 - o o o - o - o - -
 4 o - - o - o X X X o
 5 - - o - - o - o - -
 6 o X X X o - o - - o
 7 - - - - o - - o o -
 8 - o - o - o - o - -
 9 o - X X X X o - o -
Enter coordinates (X to quit): 99
Invalid coordinates! Please enter values between 0 and 9.
```

```
   0 1 2 3 4 5 6 7 8 9
 0 - - o o - o o o - -
 1 o - o - o - X - o -
 2 - o - o - - o - o o
 3 - o o o - o - o - -
 4 o - - o - o X X X o
 5 - - o - - o - o - -
 6 o X X X o - o - - o
 7 - - - - o - - o o -
 8 - o - o - o - o - -
 9 o - X X X X o - o -
Enter coordinates (X to quit): 1 7
MISS!
```

main.c battleship_log.txt ships.txt

```
Shot: 9 3 - HIT
Shot: 9 2 - HIT
Shot: 1 4 - MISS
Shot: 1 2 - MISS
Shot: 0 5 - MISS
Shot: 6 1 - HIT
Shot: 6 2 - HIT
Shot: 6 3 - HIT
Shot: 6 4 - MISS
Shot: 6 0 - MISS
Shot: 2 9 - MISS
Shot: 2 3 - MISS
Shot: 8 3 - MISS
Shot: 8 7 - MISS
Shot: 9 8 - MISS
Shot: 9 0 - MISS
Shot: 4 0 - MISS
Shot: 0 2 - MISS
Shot: 0 7 - MISS
Shot: 1 8 - MISS
Shot: 3 5 - MISS
Shot: 3 1 - MISS
Shot: 3 2 - MISS
Shot: 1 0 - MISS
Shot: 0 3 - MISS
Shot: 0 6 - MISS
Shot: 1 6 - HIT
Shot: 1 7 - MISS
Shot: 1 5 - HIT
All ships are sunk! Total shots: 53
```

main.c battleship_log.txt ships.txt

```
8 6 4
8 5 4
8 4 4
1 3 2
2 3 2
```

5. Conclusion

The program successfully simulates a Battleship game and tracks each ship hit.

Improvements such as robust ship tracking and per-hit logging were implemented.

This project provides a great learning experience in using arrays, file I/O, and game logic in C.